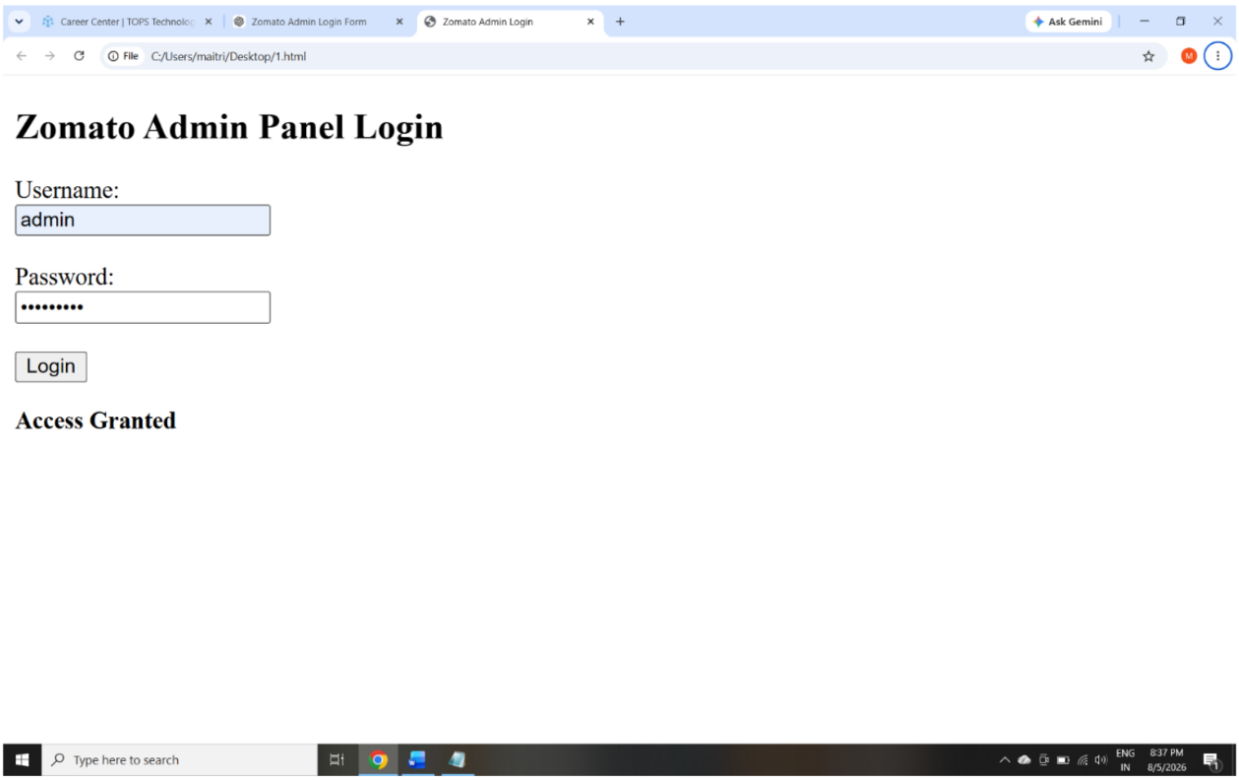
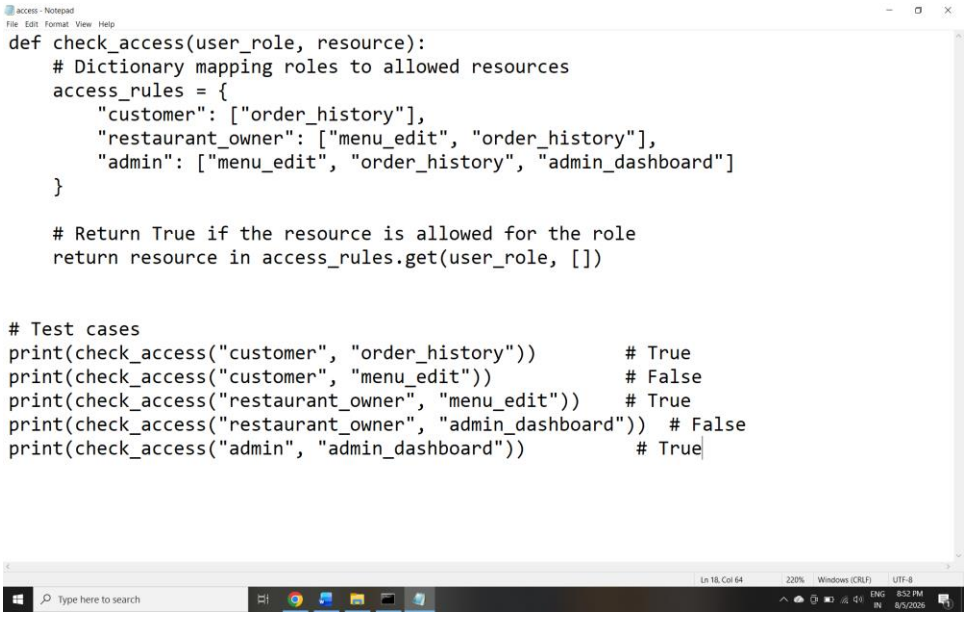
1.Create a simple login form for a mock Zomato admin panel that asks for username and password, then displays 'Access Granted' only if both fields match hardcoded admin credentials.



**2.Write a Python function check_access(user_role, resource) that returns True if the user_role is allowed to access the resource, otherwise False. Test it with roles like 'customer', 'restaurant_owner', and resources like 'menu_edit', 'order_history', 'admin_dashboard'.

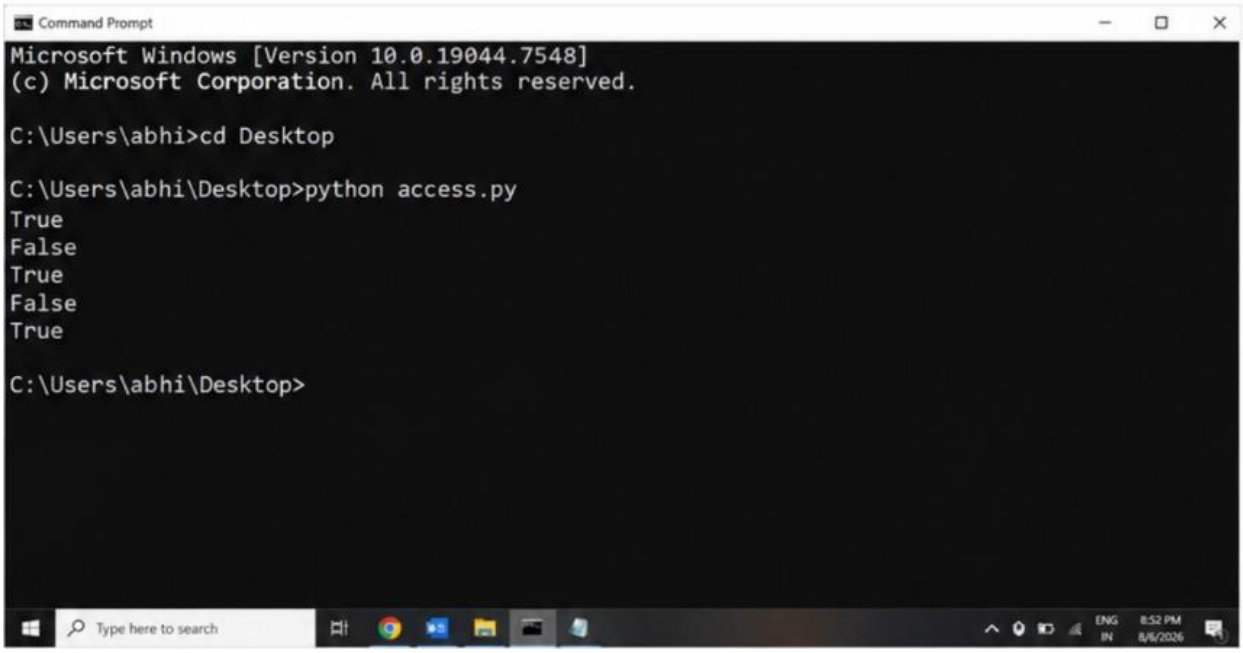
Hint: Use a dictionary to map roles to allowed resources.**



```
def check_access(user_role, resource):
    # Dictionary mapping roles to allowed resources
    access_rules = {
        "customer": ["order_history"],
        "restaurant_owner": ["menu_edit", "order_history"],
        "admin": ["menu_edit", "order_history", "admin_dashboard"]
    }

    # Return True if the resource is allowed for the role
    return resource in access_rules.get(user_role, [])


# Test cases
print(check_access("customer", "order_history"))        # True
print(check_access("customer", "menu_edit"))            # False
print(check_access("restaurant_owner", "menu_edit"))    # True
print(check_access("restaurant_owner", "admin_dashboard"))  # False
print(check_access("admin", "admin_dashboard"))          # True
```



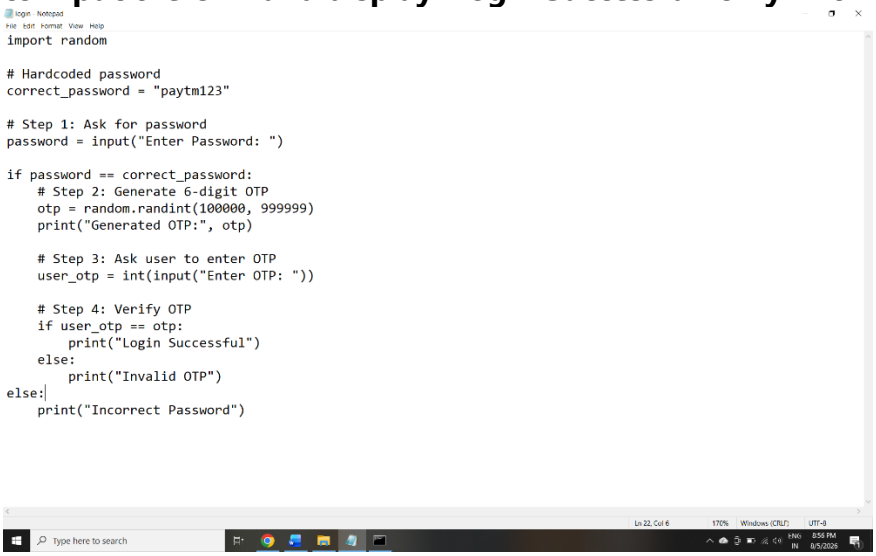
```
Command Prompt
Microsoft Windows [Version 10.0.19044.7548]
(c) Microsoft Corporation. All rights reserved.

C:\Users\abhi>cd Desktop

C:\Users\abhi\Desktop>python access.py
True
False
True
False
True

C:\Users\abhi\Desktop>
```

3.Simulate a two-factor authentication flow like Paytm: after a user enters the correct password, generate a 6-digit OTP and display it in the console. Ask the user to input the OTP and display 'Login Successful' only if it matches.



```
import random

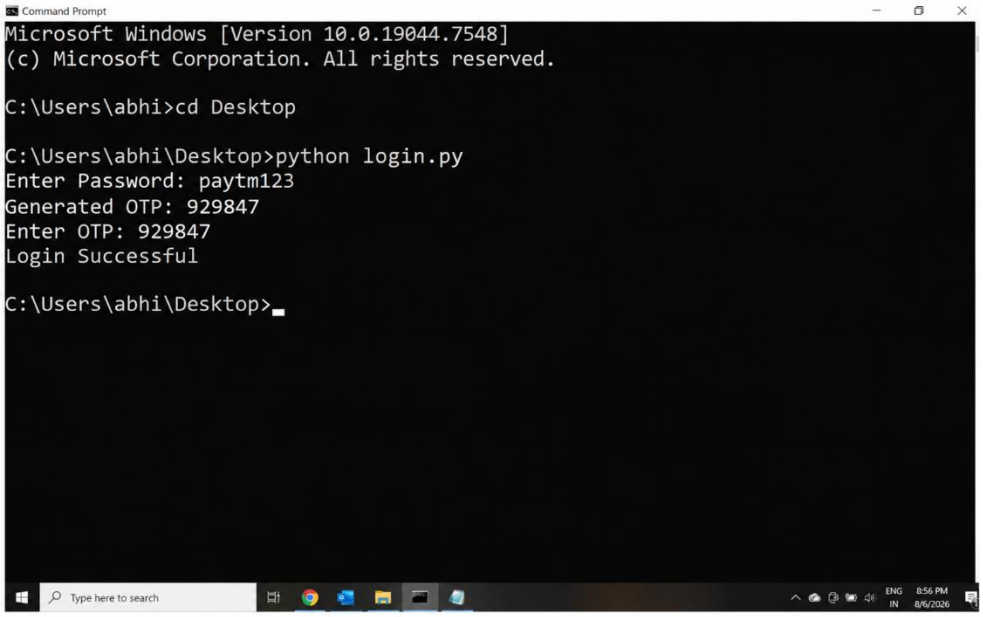
# Hardcoded password
correct_password = "paytm123"

# Step 1: Ask for password
password = input("Enter Password: ")

if password == correct_password:
    # Step 2: Generate 6-digit OTP
    otp = random.randint(100000, 999999)
    print("Generated OTP:", otp)

    # Step 3: Ask user to enter OTP
    user_otp = int(input("Enter OTP: "))

    # Step 4: Verify OTP
    if user_otp == otp:
        print("Login Successful")
    else:
        print("Invalid OTP")
else:
    print("Incorrect Password")
```



```
Microsoft Windows [Version 10.0.19044.7548]
(c) Microsoft Corporation. All rights reserved.

C:\Users\abhi>cd Desktop

C:\Users\abhi\Desktop>python login.py
Enter Password: paytm123
Generated OTP: 929847
Enter OTP: 929847
Login Successful

C:\Users\abhi\Desktop>
```

**4.Given an array of user objects with properties: username, isPremium, and lastLogin, write a function that returns only those users who are premium and have logged in within the last 7 days.

Constraint: Assume lastLogin is a date string in 'YYYY-MM-DD' format, and today is '2024-06-10'.**

user - Notepad

File Edit Format View Help

```
from datetime import datetime

def recent_premium_users(users):
    today = datetime.strptime("2024-06-10", "%Y-%m-%d")
    result = []

    for user in users:
        login_date = datetime.strptime(user["lastLogin"], "%Y-%m-%d")
        days = (today - login_date).days

        if user["isPremium"] and days <= 7:
            result.append(user)

    return result


# Example array of users
users = [
    {"username": "amit", "isPremium": True, "lastLogin": "2024-06-09"},
    {"username": "neha", "isPremium": False, "lastLogin": "2024-06-08"},
    {"username": "rahul", "isPremium": True, "lastLogin": "2024-05-30"},
    {"username": "priya", "isPremium": True, "lastLogin": "2024-06-05"}
]

print(recent_premium_users(users))
```

Ln 25, Col 35 170% Windows (CRLF) UTF-8

type here to search ENG IN 9:00 PM 8/5/2026

Command Prompt

```
Microsoft Windows [Version 10.0.19044.7548]
(c) Microsoft Corporation. All rights reserved.

C:\Users\abhi>cd Desktop

C:\Users\abhi\Desktop>python user.py
[{'username': 'amit', 'isPremium': True, 'lastLogin': '2024-06-09'}, {'username': 'priya', 'isPremium': True, 'lastLogin': '2024-06-05'}]

C:\Users\abhi\Desktop>
```

5.Use ChatGPT to generate a list of 5 common access control mistakes developers make in real-world apps like Instagram or Flipkart, and for each, write a one-line fix or prevention tip.

Sr. No.	Access Control Mistake	One-line Fix / Prevention Tip
1	Using only client-side role checks, such as hiding admin buttons in the UI.	Always verify permissions on the server before processing a request.
2	Allowing users to access another user's data by changing an ID in the URL.	Verify that the requested resource belongs to the logged-in user before returning it.
3	Giving all logged-in users admin-level access.	Use Role-Based Access Control (RBAC) and assign only the required permissions.
4	Not checking permissions on API endpoints.	Protect every API endpoint with proper authentication and authorization checks.
5	Keeping expired login sessions active.	Set session expiration times and invalidate tokens after logout or inactivity.